

电子系统导论实验报告

实验 9 【八字绕桩】



指导教师: 胡来归

学生姓名: 徐锦云 学生姓名: 周熙 学生姓名: 敖艺珊

学 号: 学 号: 学 号: _

23307130447 23307130489 23307130507

专 业: 技术科学 专 业: 技术科学 专 业: 技术科学

试验班 试验班 试验班

日 期: 2024.5.30

一、实验目的:

- 1、口在赛道中间随机设置障碍物
- 2、PID 控制电机驱动 0
- 3、提供地面引导线
- 4、通过超声、摄像头等传感器完成 8 字绕桩前行(不一定需要严格循线走,但绕行方向必须一致)

二、实验原理:

1、硬件部分:

(1) 树莓派: 树莓派是一种基于 ARM 处理器的微型计算机, 由英国的树莓派基金会开发。它拥有丰富的 GPIO (通用输入输出) 引脚, 可以用于连接各种外部设备。除了 GPIO, 树莓派还具有 USB 端口、HDMI 输出、以太网接口等, 使其成为一个功能强大的嵌入式系统。

(2) GPIO 引脚: GPIO 引脚是树莓派上用于连接外部电子设备的通用引脚。通过 GPIO 引脚, 可以实现与外部电路的通信和控制, 如连接电机、传感器等。

(3) 电机驱动: 电机驱动模块将树莓派的输出信号转换为电机可以理解的信号, 控制电机的转速和方向。使用 PWM 技术可以调整电机的转速, 通过改变脉冲宽度来控制电机的速度和方向。

(4) 摄像头: 摄像头模块用于实时获取小车行驶路径的图像数据。摄像头采集到的图像数据将被送入计算机进行图像处理和分析, 以实现车辆的自动导航功能。

2、软件部分:

(1) OpenCV 库: OpenCV (Open Source Computer Vision Library) 是一个开源的计算机视觉库, 提供了丰富的图像处理和分析功能。OpenCV 用于图像处理和分析, 包括颜色空间转换、图像分割、形态学操作等。

(2) PID 算法: PID (Proportional-Integral-Derivative) 是一种经典的控制算法, 常用于控制系统中的闭环控制。PID 算法根据摄像头获取的图像信息, 计算车辆当前位置与预期位置之间的偏差, 并调整电机的输出信号, 使车辆尽量保持在预期路径上行驶。

(3) 图像处理: 图像处理部分对摄像头获取的图像数据进行处理, 以提取出车辆行驶路径的信息。利用颜色分割和形态学操作等技术, 将摄像头采集到的图像数据中的橙色部分提取出来, 并进行后续处理, 以便后续的路径规划和导航。

3、算法流程:

(1) 初始化: 初始化 GPIO 引脚模式和初始参数, 准备好摄像头。

(2) 图像处理: 获取摄像头图像, 进行颜色空间转换和图像分割, 提取出车辆行驶路径的信息。

(3) 计算偏差: 根据处理后的图像, 计算车辆当前位置与预期位置的偏差。

(4) PID 调节: 根据偏差, 利用 PID 算法计算出电机的输出调节量, 使车辆尽量保持在预期路径上。

(5) 控制电机: 根据 PID 调节量, 调整电机的输出, 控制车辆的速度和方向, 使其跟随预期路径行驶。

(6) 循环执行: 不断重复以上步骤, 实现车辆的自动导航功能。

(7) 实验原理分析: 它基于计算机视觉和控制理论, 利用摄像头获取实时图像数据, 通过图像处理技术提取出车辆行驶路径的信息, 然后通过 PID 控制算法实现对车辆行驶方向和速度的精确控制。整个系统具有实时性和自适应性, 能够在不同环境下实现对车辆的有效控制和导航。通过软硬件的协作, 实现了小车的自动循迹功能。

三、 实验内容:

1、实物图:



2、代码

```
1  # 导入所需库
2  import RPi.GPIO as GPIO # 用于与树莓派的GPIO引脚通信
3  import time # 提供延时功能
4  import cv2 # 用于处理图像
5  import numpy as np # 提供对数组和矩阵的支持
6
7  # 定义一个函数, 用于返回参数的符号
8  def sign(x):
9      if x > 0:
10         return 1.0
11     else:
12         return -1.0
13
14  # 定义引脚
15  EA, I2, I1, EB, I4, I3 = (13, 19, 26, 16, 20, 21)
16  FREQUENCY = 50 # PWM频率
17
18  # 设置GPIO模式
19  GPIO.setmode(GPIO.BCM)
20
21  # 设置GPIO引脚为输出
22  GPIO.setup([EA, I2, I1, EB, I4, I3], GPIO.OUT)
23  GPIO.output([EA, I2, EB, I3], GPIO.LOW)
24  GPIO.output([I1, I4], GPIO.HIGH)
25
26  # 初始化PWM波
27  pwma = GPIO.PWM(EA, FREQUENCY)
28  pwmb = GPIO.PWM(EB, FREQUENCY)
29  pwma.start(0)
30  pwmb.start(0)
31
32  # 定义目标中心
33  center_now = 320
34
35  # 打开摄像头, 设置图像尺寸和格式
36  cap = cv2.VideoCapture(0)
37
38  # 初始化PID参数和误差
39  error = [0.0] * 3
40  adjust = [0.0] * 3
41  kp = 1.85
42  ki = 0.0
43  kd = 0.38
44  target = 320
45
46  # 读取一帧图像, 因为我们砸了车,
47  # 摄像头突然少读一帧, 这是为了避免误判和确认起步位置合适
48  ret, frame = cap.read()
49
50  # 转换图像颜色空间为HSV
51  hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
52
53  # 提取橙色部分
54  lower_orange = np.array([0, 50, 50])
55  upper_orange = np.array([25, 255, 255])
56  mask = cv2.inRange(hsv, lower_orange, upper_orange)
57
58  # 形态学操作去除噪声
59  kernel = np.ones((3, 3), np.uint8)
60  opening = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
61  closing = cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel)
62
63  # 将橙色部分转换为黑色
64  dst = cv2.bitwise_not(closing)
65  dst = cv2.dilate(dst, None, iterations=2)
66
```

```

67 # 在屏幕上显示图像
68 cv2.imshow("镜头画面", dst)
69
70 # 初始化黑色像素计数器
71 black_count = np.sum(dst[380] == 0)
72
73 # 打印黑色像素数量
74 print(black_count)
75
76 # 准备完毕, 等待用户按下Enter键开始运行
77 print("准备完毕! 按下Enter启动!")
78 input()
79
80 # 循环运行直到满足条件退出
81 n = 0
82 try:
83     while n < 15:
84         # 这里是防止车头起步翘起, 因此先给一个合适的加速度
85         ret, frame = cap.read()
86         hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
87         lower_orange = np.array([0, 50, 50])
88         upper_orange = np.array([25, 255, 255])
89         mask = cv2.inRange(hsv, lower_orange, upper_orange)
90         kernel = np.ones((3, 3), np.uint8)
91         opening = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
92         closing = cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel)
93         dst = cv2.bitwise_not(closing)
94         dst = cv2.dilate(dst, None, iterations=2)
95         black_count = np.sum(dst[380] == 0)
96         cv2.imshow("镜头画面", dst)
97         # 相当于加速到原来的值
98         pwma.ChangeDutyCycle(rspeed * (0.5 + n / 30))
99         pwmb.ChangeDutyCycle(rspeed * (0.5 + n / 30))
100         n += 1
101

```

```

102 while True:
103     # 呈现图像, 灰度图, 同时将橙色线路变成黑色, 别的都变成白色
104     ret, frame = cap.read()
105     hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
106     # 只呈现橙色的部分, 别的一律不在范围内, 以排除干扰, 同时又加入高斯降噪去除了多噪点
107     lower_orange = np.array([0, 50, 50])
108     upper_orange = np.array([25, 255, 255])
109     mask = cv2.inRange(hsv, lower_orange, upper_orange)
110     kernel = np.ones((3, 3), np.uint8)
111     opening = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
112     closing = cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel)
113     dst = cv2.bitwise_not(closing)
114     dst = cv2.dilate(dst, None, iterations=2)
115     black_count = np.sum(dst[380] == 0)
116     cv2.imshow("镜头画面", dst)
117
118     if black_count == 0:
119         continue
120
121     # 展示偏差
122     center_now = (np.where(dst[380] == 0)[0][0] + np.where(dst[380] == 0)[0][-1]) / 2
123     direction = center_now - 320
124     print("偏差:", direction)
125
126     # 计算偏差
127     error[0] = error[1]
128     error[1] = error[2]
129     error[2] = center_now - target
130
131     # 计算调整量
132     adjust[0] = adjust[1]
133     adjust[1] = adjust[2]
134     adjust[2] = adjust[1] + kp * (error[2] - error[1]) + ki * error[2] + kd * (error[2] - 2 * error[1] + error[0])
135     print(adjust[2])
136

```

```
138 #本想在此处加入判断交叉点直行的逻辑,但因小车速度本来就较大,完全可以凭借惯性冲过去,就没有加
139 #如果要加,逻辑就是当视野中像素点陡然增加,就认定其到达交叉点,这主要和adjust[1]和adjust[2]有关
140 #大致代码如下,没有调参数,此部分代码不完善,测试时未使用:
141 #if abs(adjust[2]) > control and abs(adjust[1]) < 20:
142 #    pwma.ChangeDutyCycle(rspeed)
143 #    pwmb.ChangeDutyCycle(lspeed)
144 # 饱和输出限制在control绝对值之内
145 if abs(adjust[2]) > control:
146     adjust[2] = sign(adjust[2]) * control
147     print(adjust[2])
148
149 # 执行PID
150
151 # 右转
152 if adjust[2] > 30:
153     pwma.ChangeDutyCycle(rspeed - 1.17*adjust[2])
154     pwmb.ChangeDutyCycle(lspeed)
155
156 # 左转
157 elif adjust[2] < -30:
158     pwma.ChangeDutyCycle(rspeed)
159     pwmb.ChangeDutyCycle(lspeed + 1.17*adjust[2])
160
161 # 直行
162 else:
163     pwma.ChangeDutyCycle(rspeed)
164     pwmb.ChangeDutyCycle(lspeed)
165
166 if cv2.waitKey(1) & 0xFF == ord('q'):
167     break
168 except KeyboardInterrupt:
169     print("结束!")
170     pass

171
172 # 释放清理
173 cap.release()
174 cv2.destroyAllWindows()
175 pwma.stop()
176 pwmb.stop()
177 GPIO.cleanup()
```

四、设计思路:

本实验基于树莓派 (Raspberry Pi) 和 Python 语言,结合了硬件控制 (使用 RPi.GPIO 库) 和图像处理 (使用 OpenCV 库)。目标是通过图像识别 (识别橙色线路) 实现小车的自动驾驶,并采用 PID 控制算法来调整小车的运动轨迹。

1、硬件设置

- ①树莓派 GPIO 引脚用于连接小车的电机控制器,包括左右两个电机的控制。
- ②采用 PWM (脉冲宽度调制) 控制电机速度,以实现精确的加速和减速。

2、图像处理

- ①小车搭载摄像头,通过 OpenCV 库捕获实时图像,并进行色彩空间转换 (从 BGR 到 HSV)。
- ②使用 HSV 色彩空间更容易提取出目标颜色 (橙色) 的物体。

3、图像识别与路径规划

①通过设置颜色阈值,利用 `cv2.inRange()` 函数提取出图像中的橙色部分,以便识别路径。

②通过形态学操作(开操作和闭操作)去除噪声,确保提取的橙色部分清晰可见。

③将橙色部分转换为黑色,其他部分转换为白色,以便后续处理。

4、PID 控制算法

①使用 PID 控制算法来调整小车的行驶方向,以使其保持在预定的路径上。

②设置 PID 参数(比例系数 k_p 、积分系数 k_i 和微分系数 k_d)来调节算法的响应速度和稳定性。

③计算偏差值,并根据偏差值调整电机的转速,使小车朝向路径中心运动。

5、实验设计

实验分为两个阶段:

①第一阶段:启动阶段,小车逐渐加速至目标速度,以适应起步状况。

②第二阶段:小车根据图像识别结果和 PID 控制算法调整行驶方向,保持在预定路径上。

6、实验步骤

①初始化 GPIO 引脚和 PWM 波。

②启动摄像头,捕获实时图像,并进行色彩空间转换和图像处理。

③根据图像识别结果,计算路径中心的偏差,并应用 PID 控制算法进行调整。

④根据 PID 输出调整电机的转速,以实现小车的转向。

⑤循环执行以上步骤,直到满足退出条件(例如按下'q'键)。

7、实验分析

①小车起步阶段采用渐变加速的方式,以避免起步时车头翘起的情况。

②图像识别阶段,通过提取出的橙色路径部分,计算路径中心的偏差,并根据偏差值调整小车的行驶方向。

③PID 控制算法通过比例、积分和微分的组合来实现对小车行驶方向的精确调节。

④实验中使用的阈值、形态学操作和 PID 参数需要根据实际场景进行调整,以获得最佳的性能和稳定性。

五、总结与思考:

在此次试验中,我们小组的整个过程可以分为硬件安装和代码调试两个部分,在这两部分中我们都有诸多收获:

(一) 硬件安装:

1、小车运行方向,一开始代码中的左右轮与我们实际的左右轮相反,所以在巡迹过程中,小车识别到向右转的弯却出现向左一直绕圈的情况。对于这一问题,我们在调试过程中学习到了可以将代码中控制左右轮的引脚编号调换,小车就恢复了正常识别弯道并做出正确的反应。之后,小车巡迹并不稳定,我们还是采取

了将万向轮作为车头,这样小车在行进过程中也更加稳定。

2、在研究调参之后,我们得到一组相对稳定的参数,应对这一参数不能持续稳定,我们又总结出这个参数回对应一个区间的合适的电量。

3、考试当天小车被砸到地上,我们发现这一参数又不能正常运行了,应对这一危机,我们选择重调参数,在这一过程中,我们发现小车摆动很大,我们排除了代码问题后,成功总结出是由于我们的摄像头太低,以至于小车识别的轨迹太粗从而不稳定所导致,可以调高摄像头解决摆动的问题。

第二天,我们发现了新的问题。

- (1) 车头起步翘起,因此先给一个合适的加速度,先让小车稳定直行一段时间,通过交叉口,且速度加上来,然后再进入正式的循迹模式。
- (2) 摄像头噪点急剧增加,换摄像头来不及,于是加入一个高斯模糊,起到膨胀腐蚀的作用。
- (3) 本想在代码中加入判断交叉点直行的逻辑,但因小车速度本来就较大,完全可以凭借惯性冲过去,就没有加。如果要加,逻辑就是当视野中像素点陡然增加,就认定其到达交叉点,这主要和 `adjust[1]`和 `adjust[2]`有关。但是此部分代码不完善,测试时未使用。
- (4) 小车转大弯的时候,通过微调 `adjust[2]`前的系数,可以解决问题。

4、小车启动时翘头从而导致小车偏离轨道,就牵涉到小车重心问题,可在车头加上适当的配重,解决这一问题。

(二) 代码调试(排除硬件问题后)

- 1、初始代码中将 `pid` 都设置为接近于 0 的数值;
- 2、若小车直线走不直(即不循迹),逐渐增大参数 `p`,大致每次增加 0.1,到合适参数附近后变成每次增加 0.02~0.05;
- 3、若小车跑动摆动较大,逐渐增大参数 `d`,大致每次增加 0.1,到合适参数附近后变成每次增加 0.02~0.05;
- 4、若转弯时幅度过大,适当增加一点点转弯系数,增加过多的转弯系数会让小车转弯时速度减慢过多,影响考核的圈数。
- 5、影像处理时噪点过多,可加入降噪处理的代码,以及可以提高代码中颜色识别的精度,给出更精确的 RGB。